SYSTEM DESIGN & ARCHITECTURE JOBS_AUTO_APPLY V3

JD-Aware Resume Tailoring & Adaptive Apply Engine

An additive architecture that tailors each application's résumé to its job description, reuses tailored variants across similar postings, and replaces flat retry counts with a signal-fused human-intervention policy — designed to cost $0 in cloud LLM fees, minimize bot-detection risk, and minimize human interruption.

base	jobs_auto_apply v2
new modules	tailoring/ · resilience/ · platform_adapter.py
cloud LLM cost	$0 by design (local Ollama only)
companion	resume-tailoring-sda.md · IMPLEMENTATION_PROMPT.md

> **Reading guide** — Part A (§A0–A13) is the architecture. Part B (§B1–B9) is the annexure: real issues found in this project's own build-out and how v3 corrects them. A requirements-traceability table closes the document.

A0 executive summary

v2 applies to jobs with one static résumé. This is the single highest-leverage gap for ATS keyword-match and recruiter skim-relevance. v3 inserts a Resume Tailoring Subsystem between scoring and apply, backed by three enhancements: a similarity-based Variant Library (the primary cost control), a signal-fused Human-Intervention Engine (replaces blind retry counts), and a formal Platform Adapter contract (real OCP, not asserted OCP).

The tailoring pipeline is deliberately **tiered and mostly local** — not "call a cloud LLM for every job," which would be slow, costly, and a larger hallucination surface. Full design: §A4.

A1 design principles, applied concretely

PRINCIPLE	CONCRETE APPLICATION
SRP	`jd_extractor.py` only extracts. `variant_store.py` only persists/matches. `tailoring_engine.py` only generates. `ats_validator.py` only validates. None import Playwright.
OCP	New platforms implement `PlatformAdapter`. New tailoring strategies implement `TailoringStrategy`. The orchestrator never changes for either.
DIP	Orchestrator depends on interfaces, not concretes — concretes wired from config at startup.
DRY	`ats_validator.py` / `jd_extractor.py` written once, shared by all six adapters — a real dedup vs. v2's per-site duplication.
SoC	*What to say* (tailoring), *is it safe to proceed* (signal engine), *is it well-formed* (validator), and *how to click this DOM* (adapter) are four separate concerns, previously blended.
YAGNI over cargo-cult microservices	"Microservices principles" = loosely-coupled, independently-testable, independently-swappable modules — deployed as *one process*. Real network microservices for a single-user tool would add latency/cost against the stated goal. See §A9.

A2 high-level architecture

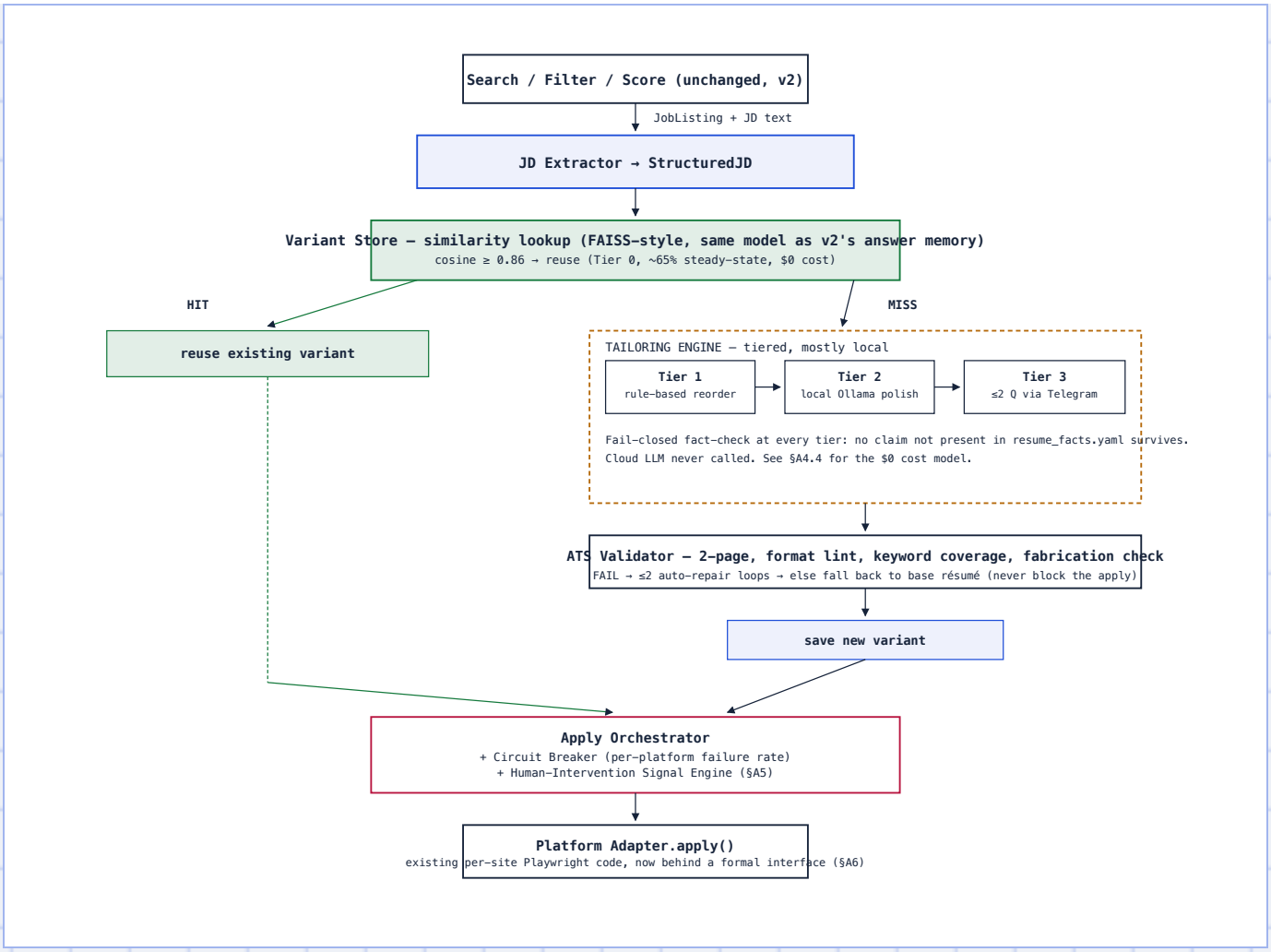


Fig. 1 — v3's high-level flow. Green = the reuse path (dominant in steady state, $0 cost). Amber = the tiered tailoring path, entered only on a variant-store miss. Red = the existing apply layer, now wrapped with a circuit breaker and signal-fused retry policy instead of a flat retry count.

A3 module decomposition — "microservices principles, monolith deployment"

Each module is a narrow-interface Python package, independently unit-testable and swappable via a registered `Protocol` implementation, communicating via in-process typed dataclasses — not HTTP/queues. See §A1's YAGNI note for why literal network services are an anti-pattern at this system's scale.

tailoring/

`jd_extractor.py` · `variant_store.py` · `engine.py` · `ats_validator.py` · `render.py` · `clarify.py`

resilience/

`signals.py` · `policy.py` (pure, no I/O) · `circuit_breaker.py`

platform_adapter.py

`PlatformAdapter` Protocol + registry — the DIP boundary between orchestrator and per-site code

cost_ledger.py + config/loader.py

Token/call accounting (SQLite) and layered base+overlay config resolution

A4 resume tailoring subsystem — deep dive

A4.1 — Why tiered, not "always call an LLM"

A cloud full-résumé rewrite costs ~$0.02–0.06/call → $2–6 per 100 applications, and is 100% avoidable for the majority of JDs, which cluster heavily around a handful of recurring requirement templates at this seniority level. The tiers, cheapest first:

1. **Tier 0 — similarity reuse.** Embed the `StructuredJD` (not raw text — tighter, more comparable) with the same local MiniLM model v2 already runs for answer-memory. Cosine-search the Variant Library; above threshold, reuse verbatim. Cost: ~5ms CPU, $0.
2. **Tier 1 — rule-based reorder + keyword injection.** No LLM. Reorders already-true content to surface JD-matching keywords earlier; injects exact-phrase keywords *only* when already present in `resume_facts.yaml` — deterministic substring/synonym match, cannot fabricate.
3. **Tier 2 — local LLM polish.** Only for requirement clusters Tier 1 can't satisfy. Scoped to one section at a time (never whole-document), on the same local Ollama model v2 already runs for screening answers. System-prompt constraint mirrors v2's existing generator discipline: never state a skill/employer/metric not in the fact files.
4. **Tier 3 — human clarification.** At most 2 batched questions via the existing Telegram plumbing, bounded wait (default 600s), safe default = omit the unresolved claim.

A4.2 — Per-job sequence

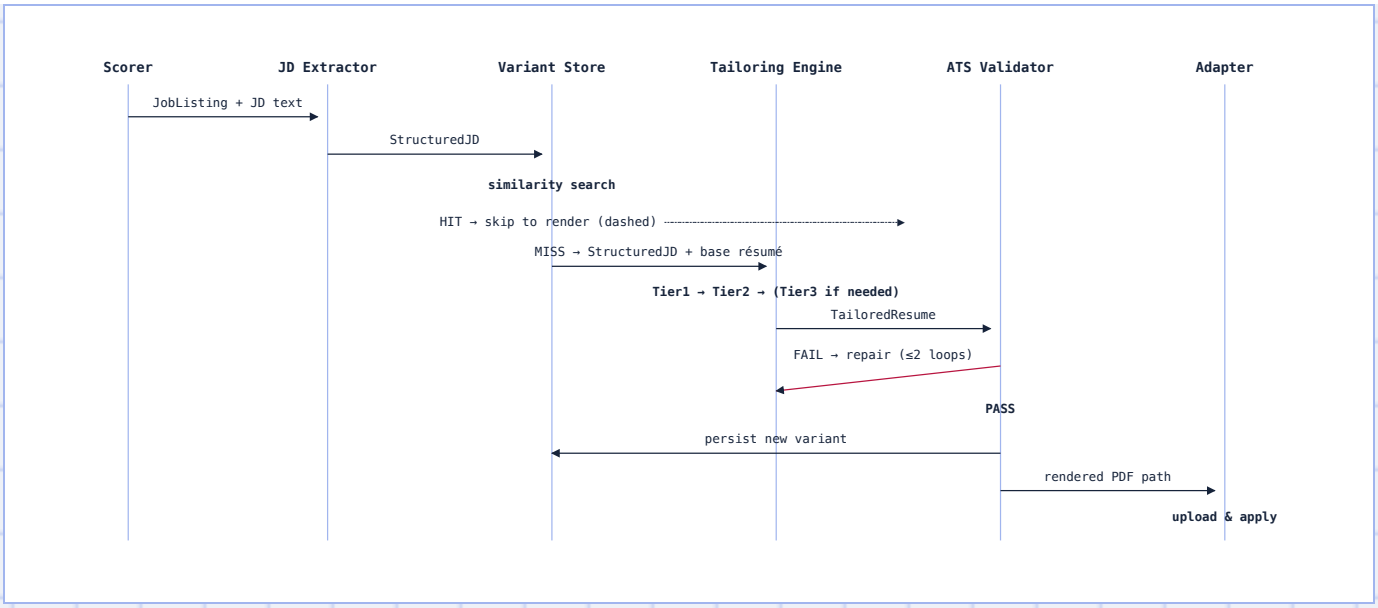


Fig. 2 — Sequence for one job. The Tier-0 reuse path (dashed green) skips the entire tailoring engine — this is the dominant path in steady state.

A4.3 — ATS-safety constraints, enforced mechanically

CONSTRAINT	ENFORCEMENT
Exactly 2 pages	Fixed single-column 11pt template; overflow trims lowest-JD-relevance bullets first; >40% underflow on page 2 is flagged, never padded with filler
ATS-parseable format	Single column, standard section headers, no tables/text-boxes/header-footer body text/icons — the documented Workday/Greenhouse/Taleo-class parser failure modes
No fabrication	Every generated claim checked against `technical_skills` / `core_skills` / `experience[]`; failing claims are stripped, not left in — fail closed
Keyword coverage	TF-based match ≥70% of must-have keywords targeted; below that, one more Tier-2 pass on the gap only
File format	PDF from structured data (WeasyPrint/ReportLab) — decoupled from Playwright per SoC, no browser dependency for rendering

A4.4 — Cost model, per 100 applications

STEADY-STATE PATH DISTRIBUTION (~65% reuse rate after ~2 weeks)

PATH	APPLICATIONS	COST/APP	SUBTOTAL
Tier 0 (reuse)	65	~5ms CPU, $0	$0
Tier 1 only	20	~50ms CPU, $0	$0
Tier 2 (local LLM)	14	2–8s local compute, $0	$0

PATH	APPLICATIONS	COST/APP	SUBTOTAL
Tier 3 (human clarify)	1	$0 (Telegram)	$0

> **Total incremental cloud LLM cost: $0.** Structural, not a prompting trick — every generative step runs on the already-owned local Ollama instance. Only cost is compute-seconds on an otherwise-idle box, and a few hundred KB/variant on disk.

A5 `adaptive human-intervention signal engine`

The direct architectural answer to: optimize retries vs. wait time, don't retry against things only a human can resolve, take the right call from signals.

A5.1 — The problem with v2's retry model

v2's `apply_retries` is a flat count with fixed backoff, blind to *why* an attempt failed. A CAPTCHA, a dead session, and a transient DOM timing miss are all "a failure," retried identically — and retrying against a CAPTCHA is itself a bot-detection signature, meaning the naive retry loop can actively cause the account-blocking outcome it should avoid.

A5.2 — Signal fusion

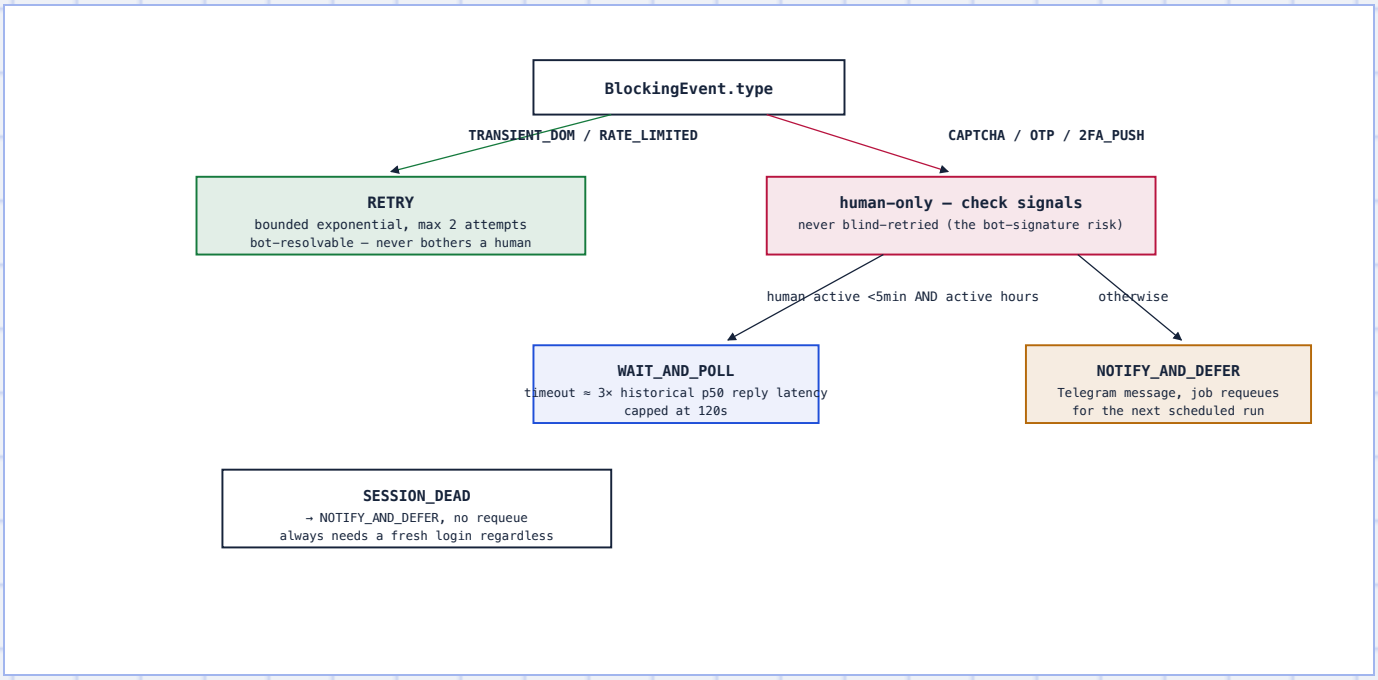


Fig. 3 — The decision policy (`resilience/policy.py`), a pure function of event type + fused signals. Zero blind retries against anything a bot

```
cannot resolve is the single rule that most directly answers "don't get
the account blocked."
```

SIGNALS FUSED PER DECISION

SIGNAL	SOURCE	WHAT IT INDICATES
`human_recent_activity_seconds`	Last SSH/RDP activity or last Telegram reply	Human plausibly at the keyboard right now
`time_of_day_in_active_hours`	Config-derived active window	Outside it, skip straight to async defer
`historical_reply_latency_p50`	Rolling stat from past Telegram turnaround	Sets a realistic wait budget instead of a guess
`challenge_type_resolvability`	Static table	Which branch even applies
`platform_recent_failure_rate`	Circuit breaker's rolling window	Backs off the whole platform, independent of this one event

A6 `platform adapter contract`

`PlatformAdapter` is a `runtime_checkable Protocol`: `collect_listings()`, `apply()`, `verify_session()` (all existing v2 behavior, now formalized), plus **new** `detect_blocking_event()`. Every v2 site package becomes a thin wrapper — *no rewrite of working Playwright DOM logic*, only the seams are formalized. Adding platform #7: implement the Protocol, register it — zero changes to the orchestrator, tailoring subsystem, or resilience engine.

A7 `cost & token budget model`

`cost_ledger.py` — one SQLite row per LLM call. Since every generative call defaults to local Ollama, cost is structurally $0; the ledger's real value is **call-count/compute-time observability** — catching a regression (e.g. one platform's verbose JDs triggering excess Tier-2 calls) before it becomes a problem.

METRIC	TARGET
Wall-clock for 100 applications	≤ 3 hours
LLM calls per 100 applications	~15–20 Tier-2 tailoring calls + existing answer-engine calls
Cloud LLM cost	$0 (local-only by default)
Human interruptions per 100 applications	< 5, only for genuinely human-only events

A8 resume variant library — storage schema

SQLite, not v2's JSON-file pattern — a deliberate exception because this table needs similarity search over structured metadata and will grow to hundreds of rows. `base_resume_version` (a hash of `resume_facts.yaml`) makes stale-variant invalidation automatic: edit your résumé facts, every old variant stops matching, no explicit cache-busting logic needed.

A9 anti-patterns explicitly avoided

ANTI-PATTERN	WHY AVOIDED HERE
God object	Split into 5 single-purpose modules, each independently testable/replaceable
Premature microservices	Interpreted as internal modularity, not network services — see §A1
Retry storm	§A5 forbids blind retry against human-only events — the largest single correction in this document, also a safety fix
Silent fabrication	Fail-closed fact-check at every tailoring tier
Unbounded LLM cost creep	LLM is the last resort in the tier order, mirroring v2's proven answer-engine waterfall
Config sprawl	§A10 fixes the 3-file duplication this project itself hit
Tight coupling to Playwright inside business logic	Tailoring subsystem has zero browser imports
Global mutable "is human available" state	Centralized in `resilience/signals.py`, one ownership boundary

A10 layered configuration

Replaces the current 3 fully-duplicated ~380-line YAML files with `config/base.yaml` + a thin overlay per deployment (`local-mac.yaml` , `docker.yaml` , `aws-vm.yaml`). Overlay deep-merges onto base. A shared-content edit (e.g. `target_roles`) happens once — eliminating the drift risk this project's own three-file setup already created.

A11 security considerations

- **Prompt injection via JD text**: JD content is always framed as quoted data to analyze, never as instructions — same discipline v2 already applies to screening-question text.

- **PII in the variant library**: same sensitivity class as existing `resume.pdf` / `profile/*.yaml`; extends existing `.gitignore` patterns, no new exposure class.
- **No new secrets**: Ollama is local, no API keys introduced.
- **Fabrication as a reputational/legal risk**, arguably the highest-stakes failure mode in this feature — treated with defense-in-depth (§A4.3), not an afterthought.

A12–13 `extensibility & rollout`

New platform → implement `PlatformAdapter`. New tailoring strategy → implement `TailoringStrategy`, select via config. New output format → implement `ResumeRenderer`. Rollout: (1) adapter refactor, pure, no behavior change; (2) tailoring in shadow mode (generates + validates, still uploads the static résumé until reviewed); (3) resilience engine, feature-flagged, falls back to v2's flat retry when off; (4) go-live after a supervised dry-run, matching this project's own established "dry_run: true first" discipline.

`PART B — ANNEXURE`

v2 Issues Found During This Project, and How v3 Corrects Them

Grounded in problems this project actually hit while building and deploying v2 — not hypothetical critique.

B1 `configuration sprawl`

Observed: three ~380-line YAML files (`config.yaml`, `config.docker.yaml`, `config.vm.yaml`) sharing ~90% content, manually kept in sync — and drift *already* nearly happened during this project's own role/compensation tuning. **Root cause**: no layering concept in `config.py`. **Fix**: §A10.

B2 `flat, failure-type-blind retry policy`

Observed: `apply_retries` treats a dead browser, a CAPTCHA, and a click-miss identically. **Root cause**: designed around one dominant v2-era failure mode (DOM flakiness), never extended for bot-detection-era failure modes. **Fix**: §A5 — the largest single gap this project surfaced, and a safety fix, not just an efficiency one.

B3 no cost/token observability

Observed: policy knobs exist (`max_concurrency`, `min_confidence`) but no measurement of what they cost over time — invisible even in v2, load-bearing once tailoring adds a higher-token-cost feature. **Fix**: §A7's minimal ledger.

B4 implicit platform integration contract

Observed: six site packages called by name via a hand-maintained dict, no interface enforcement — a missing function is only caught at runtime, for that platform, mid-run. **Fix**: §A6's `runtime_checkable Protocol`, checked at registry time.

B5 Wellfound headless fragility — reproduced during this project's Docker smoke test

Observed: headless Chromium without a real Chrome profile verified as "logged in" but rendered a signup page — 0 listings, silently reported as a normal empty result. **Root cause**: `headless` is a single boolean with no per-site bot-detection-sensitivity awareness. **Fix**: `detect_blocking_event()` treats "verified but suspiciously empty" as a signal, routed through §A5 — v2's current silent-empty-result behavior actively hides this failure, which is worse than failing loudly.

B6 dependency misclassification — a hard crash this project hit and fixed

Observed: `dataclasses-jsonschema`, a hard runtime import in `config.py`, was dev-extras-only and absent from `requirements.txt` — worked by accident on the Mac, crashed immediately in Docker. **Fix**: already corrected in this project; documented here as a pattern — any new dependency must land in `requirements.txt` in the same commit, caught mechanically by the established Docker smoke-test step.

B7 host/port and env-var wiring bugs — observed and fixed this project

Observed: `serve` defaulted to `127.0.0.1:8765`; Docker assumed `0.0.0.0:8000`; `JAA_INTERVAL_MINUTES` was silently inert outside `--reload` mode. **Root cause**: two parallel, never-reconciled configuration paths. **Fix**: not a v3 architecture change (fixed directly), but §A10's single source of truth structurally prevents this bug class recurring.

B8 single Chrome-profile serialization — an unstated scalability ceiling

Observed: real-Chrome-profile reuse (now used for all 6 platforms on the AWS VM) means only one process may hold the profile at a time — correct, but never documented as a permanent throughput ceiling. **Fix**: documented explicitly here; scaling past it means either accepting §B5's fragility for cookie-based platforms, or running multiple identities — explicitly not recommended (compounds ToS/detection risk).

B9 fabrication risk — net-new, designed against up front

Not a v2 bug (v2 never generates résumé content) — included because v2's "never guess" answer-engine philosophy had to be *deliberately re-derived* for résumé prose, a meaningfully higher-stakes generation surface than selecting among pre-validated screening answers. See §A4.3, §A11.

REQUIREMENTS TRACEABILITY

REQUIREMENT	SECTION
JD-aware tailoring, 1–2 clarifying questions	§A4.1 (Tier 3), §A4.2
Exactly 2 pages, ATS-friendly	§A4.3
Store/reuse tailored resumes for similar JDs	§A4.1 (Tier 0), §A8
Scalable and extensible	§A3, §A6, §A12
Safe/secure, low LLM cost	§A11, §A4.4, §A7
Minimum retries, optimized time/RAM/CPU	§A5, §A4.4
SOLID/DRY/SRP/OCP/SoC, modularity	§A1, §A3, §A6
Avoid anti-patterns	§A9

REQUIREMENT	SECTION
Not get blocked by portals	§A5.3–4, §B5
Maximize shortlisting/callback chances	§A4 (whole)
Optimize cost/time for 100 applications	§A4.4, §A7
Minimize human intervention, right retry-vs-wait call from signals	§A5 (whole)